

Name: _____ Register No: _____

Experiment 1.a

Question: Analyze the trend of data science job postings over the last decade for the given data on the number of data science job postings each year. Use pandas for data manipulation and matplotlib/seaborn for visualization.

Code

```
import json
import re
from collections import Counter

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

def make_jobs_data():
    rng = np.random.default_rng(42)
    years = list(range(2015, 2026))
    roles = ["Data Analyst", "Data Engineer", "Data Scientist",
             "ML Engineer", "BI Developer"]
    rows = []
    for y in years:
        for r in roles:
            base = {"Data Analyst": 500, "Data Engineer": 350, "Data Scientist": 400,
                    "ML Engineer": 150, "BI Developer": 250}[r]
            count = int(base * (1 + 0.18 * (y - 2015)) + rng.integers(-40, 40))
            rows.append({"Year": y, "Role": r, "Postings": count})
    return pd.DataFrame(rows)

def exp_1a():
    df = make_jobs_data()
    yearly = df.groupby("Year")["Postings"].sum().reset_index()
    print(yearly)

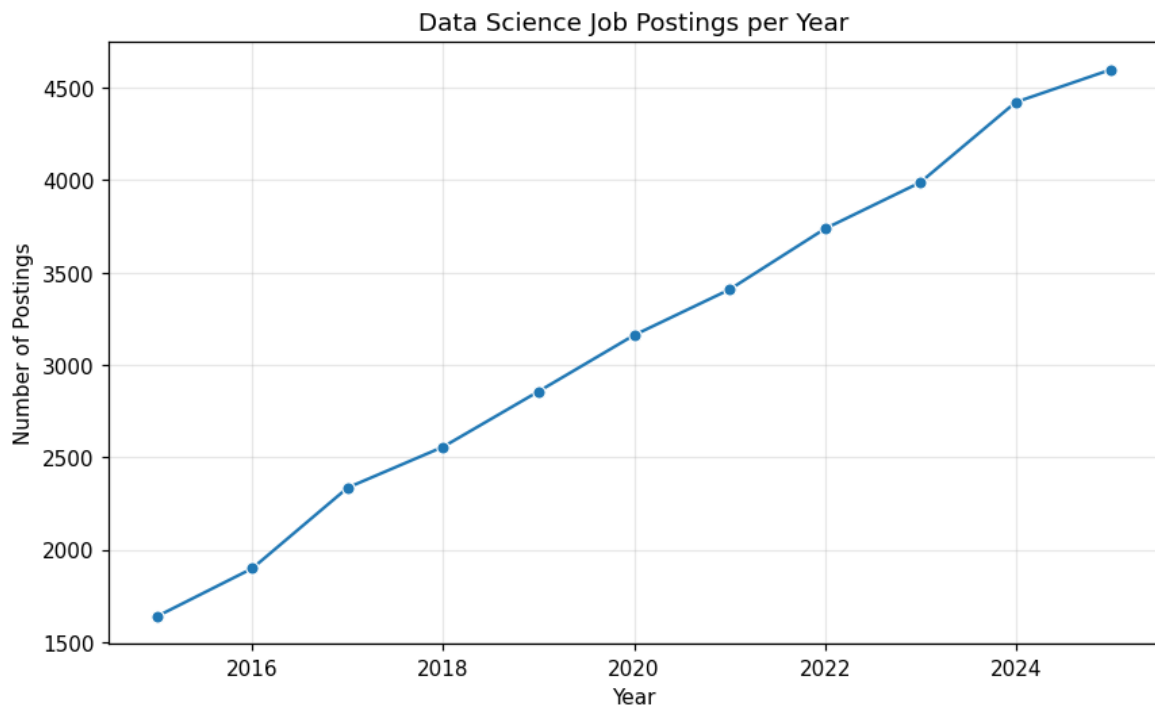
    plt.figure(figsize=(8, 5))
    sns.lineplot(data=yearly, x="Year", y="Postings", marker="o")
    plt.title("Data Science Job Postings per Year")
    plt.xlabel("Year")
    plt.ylabel("Number of Postings")
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()

exp_1a()
```

Output

	Year	Postings
0	2015	1639
1	2016	1899
2	2017	2336
3	2018	2557
4	2019	2857
5	2020	3161
6	2021	3408
7	2022	3737
8	2023	3988
9	2024	4422
10	2025	4599

Graph



Experiment 1.b

Question: Analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from a dataset.

Code

```
import json
import re
from collections import Counter

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

def make_jobs_data():
    rng = np.random.default_rng(42)
    years = list(range(2015, 2026))
    roles = ["Data Analyst", "Data Engineer", "Data Scientist",
             "ML Engineer", "BI Developer"]
    rows = []
    for y in years:
        for r in roles:
            base = {"Data Analyst": 500, "Data Engineer": 350, "Data Scientist": 400,
                    "ML Engineer": 150, "BI Developer": 250}[r]
            count = int(base * (1 + 0.18 * (y - 2015)) + rng.integers(-40, 40))
            rows.append({"Year": y, "Role": r, "Postings": count})
    return pd.DataFrame(rows)

def exp_1b():
    df = make_jobs_data()
    role_totals = df.groupby("Role")["Postings"].sum().sort_values(ascending=False)
    print(role_totals)

    fig, ax = plt.subplots(1, 2, figsize=(12, 5))
    sns.barplot(x=role_totals.index, y=role_totals.values, ax=ax[0])
    ax[0].set_title("Postings by Role")
    ax[0].tick_params(axis="x", rotation=30)

    ax[1].pie(role_totals.values, labels=role_totals.index, autopct="%1.1f%%")
```

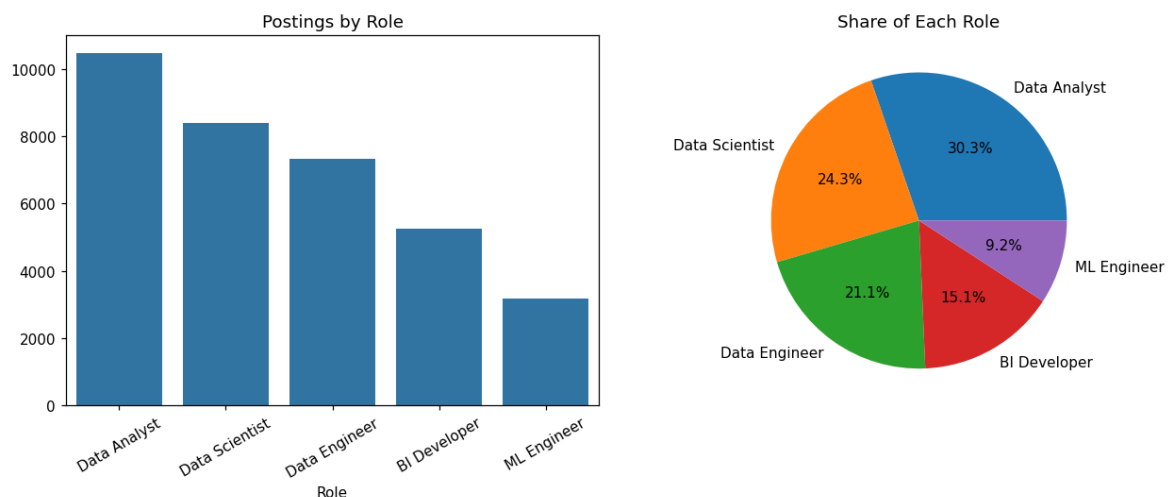
```
ax[1].set_title("Share of Each Role")
plt.tight_layout()
plt.show()
```

exp_1b()

Output

```
Role
Data Analyst      10470
Data Scientist    8402
Data Engineer     7316
BI Developer      5242
ML Engineer       3173
Name: Postings, dtype: int64
```

Graph



Experiment 1.c

Question: Conduct an experiment to differentiate Structured, Un-structured and Semi-structured data based on data sets given.

Code

```
import json
import re
from collections import Counter

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

def exp_1c():
    # Structured: rows and columns with a fixed schema (table / CSV / SQL)
    structured = pd.DataFrame({"id": [1, 2, 3],
                               "name": ["Asha", "Ravi", "Meena"],
                               "marks": [88, 92, 79]})
    print("STRUCTURED (fixed schema, tabular)")
    print(structured, "\n")

    # Semi-structured: has tags/keys but no rigid schema (JSON / XML)
    json_text = """
    [{"id": 1, "name": "Asha", "skills": ["Python", "SQL"]},
     {"id": 2, "name": "Ravi", "address": {"city": "Chennai", "pin": 600001}},
     {"id": 3, "name": "Meena"}]
    """
    semi = json.loads(json_text)
    print("SEMI-STRUCTURED (JSON: keys differ per record)")
    for rec in semi:
        print(rec)
    print("Flattened with pandas:\n", pd.json_normalize(semi), "\n")
```

```
# Unstructured: free text / images / audio, no predefined model
unstructured = ("Customer said the delivery was late but the product quality "
               "was excellent. Will buy again!")
print("UNSTRUCTURED (free text)")
print(unstructured)
print("Word count:", len(unstructured.split()))

summary = pd.DataFrame({
    "Type": ["Structured", "Semi-structured", "Unstructured"],
    "Example": ["CSV, SQL table", "JSON, XML", "Text, images, audio"],
    "Schema": ["Fixed", "Flexible (tags/keys)", "None"],
    "Python tool": ["pd.read_csv", "json / json_normalize", "NLP / string ops"],
})
print("\n", summary.to_string(index=False))
```

exp_1c()

Output

STRUCTURED (fixed schema, tabular)

	id	name	marks
0	1	Asha	88
1	2	Ravi	92
2	3	Meena	79

SEMI-STRUCTURED (JSON: keys differ per record)

```
{'id': 1, 'name': 'Asha', 'skills': ['Python', 'SQL']}
{'id': 2, 'name': 'Ravi', 'address': {'city': 'Chennai', 'pin': 600001}}
{'id': 3, 'name': 'Meena'}
```

Flattened with pandas:

	id	name	skills	address.city	address.pin
0	1	Asha	[Python, SQL]	NaN	NaN
1	2	Ravi	NaN	Chennai	600001.0
2	3	Meena	NaN	NaN	NaN

UNSTRUCTURED (free text)

Customer said the delivery was late but the product quality was excellent. Will buy again!
Word count: 15

	Type	Example	Schema	Python tool
Structured	CSV, SQL table	Fixed	pd.read_csv	
Semi-structured	JSON, XML	Flexible (tags/keys)	json / json_normalize	
Unstructured	Text, images, audio	None	NLP / string ops	

Experiment 1.d

Question: Conduct an experiment to encrypt and decrypt given sensitive data. Use the cryptography library Fernet.

Code

```
import json
import re
from collections import Counter

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

def exp_1d():
    from cryptography.fernet import Fernet

    key = Fernet.generate_key()          # 1. generate secret key
    cipher = Fernet(key)                 # 2. create cipher object
    print("Key:", key.decode())

    data = "Name: Asha, Aadhaar: 1234 5678 9012, Salary: 75000"
    token = cipher.encrypt(data.encode()) # 3. encrypt (bytes in, bytes out)
    print("Encrypted:", token.decode())

    original = cipher.decrypt(token).decode() # 4. decrypt
    print("Decrypted:", original)
    assert original == data
    # Wrong key cannot decrypt
```

```
try:
    Fernet(Fernet.generate_key()).decrypt(token)
except Exception as e:
    print("Decryption with wrong key failed:", type(e).__name__)
```

exp_1d()

Output

Key: wfp5s-EjUyXrhNaITv8KppaZz2An3u_Y_4qLhVV0lCg=
Encrypted: gAAAAABqweZd-evGASmB6I72IcAKA7sEqfJf2BPqEcMXctbXhNc8-
pBN6U4SPryZaMuhasUBWo72eW2kkyL5yH63DELDI06yFn6P2uD_iE1vhiTA8ejSuirJknzfl-W_-
xuNU1oMZHRuEXJ6YTf0vLrQRNvgww9_ow==
Decrypted: Name: Asha, Aadhaar: 1234 5678 9012, Salary: 75000
Decryption with wrong key failed: InvalidToken